

TESTING DOCUMENT

DACS3203 • Secure Software Development

Movie Theatre Management System

Yaser, Al-Mulaiky – 60105056

Omar, Suhail - 60104487

Testing report for peer review, SAST, and DAST

Prepared by	Project group
Course	DACS3203
Date	7 April 2026

1. Peer Review

How it was checked. The review was done for the main files. After each accepted change, the app was run again and the related screens were tested to make sure the normal behavior was still working.

Fix 1 – Safer database cleanup in DBUtils.java

Issue found. The `closeConnection(Connection con, Statement stmt)` method was closing the statement and connection directly. If one of them was null, cleanup could fail.

Change made. Null checks were added before closing both objects. This made the cleanup safer and reduced the chance of errors when a query failed before the connection or statement was fully created.

Testing after the fix. The app was run and the normal database actions were tested, including login and opening screens that read data from the database. The system still worked normally after that.

Fix 2 – Explicit ResultSet cleanup

Issue found. Several classes used `ResultSet` objects but did not explicitly close them after reading query results.

Change made. Explicit `ResultSet` closing was added before closing the statement and connection. The affected files were `AuthenticationService.java`, `ManageMovie.java`, `ShowAllMovies.java`, `ShowAllActions.java`, `ShowAllMaintenanceReports.java`, and `MoviePopularityReport.java`.

Testing after the fix. Login was tested first to confirm `AuthenticationService.java` still worked. After that, the manager role was used to test Update Movie, Show All Movies, Show All Actions, Show All Maintenance Reports, and Movie Popularity Report. All tested screens loaded correctly and no new errors were observed.

Fix 3 – Removed unused imports

Issue found. A few classes still had imports that were no longer being used.

Change made. Unused imports were removed from `ShowAllMaintenanceReports.java`, `StaffView.java`, and `UpdateMovie.java`. This was only cleanup and did not change the logic.

Testing after the fix. The project was rebuilt after the cleanup and the application was opened again. Basic navigation was tested to confirm the system still compiled and ran normally.

Fix 4 – Corrected movie showtime check in MovieCheck.java

Issue found. The `hasShowtime(movieName)` method was checking showtime using `movieName`, while the showtime table stores `movieId`. This could make the delete movie check inaccurate.

Change made. The check was updated to use `movieId` instead of `movieName`. The method now gets the id with `getMovieIdByName(movieName)` and then checks showtime using `stmt.setInt(1, movieId)`.

Testing after the fix. Movie deletion was tested in both cases. When the movie had a showtime, the delete action was blocked. When the movie had no showtime, deletion worked correctly.

2. SAST

Security Testing & Vulnerability Assessment

Theater Management System

Introduction

This report presents the results of a security assessment conducted on the Theater Management System, a JavaFX-based desktop application integrated with a relational database.

The application includes authentication, role-based access control, movie and showtime management, ticket booking, reporting features, account lock mechanism, and action logging.

The objective is to evaluate the system against secure software development principles and identify potential vulnerabilities.

Testing Tool

Tool Used: VCG Static Code Analyzer

Testing Type: Static Application Security Testing (SAST)

The tool scans source code to detect SQL injection, input validation issues, and insecure coding practices, below is the Summary table:

[illegible]

Findings

3.1 SQL Injection (Critical)

☐ 1	Critical	Potential SQL Injection	The application appears to allow SQL injection via a pre-prepared dynamic SQL statement. No validator plug-ins were located in the application's XML files.
☐ 1	Critical	Potential SQL Injection	The application appears to allow SQL injection via a pre-prepared dynamic SQL statement. No validator plug-ins were located in the application's XML files.
☐ 1	Critical	Potential SQL Injection	The application appears to allow SQL injection via a pre-prepared dynamic SQL statement. No validator plug-ins were located in the application's XML files.

The tool flagged these three as potential SQL Injection vulnerabilities, which is a **false positive**, since these 3 classes do not directly use untrusted user input or modify the database (INSERT / UPDATE / DELETE) but only used for internal reporting and aggregation purposes, and the SQL structure is not modified by external user-controlled data.

3.2 Input Validation Issues (Standard)

4	Standard	getResourceAsStream	This function acts as an entry point for external data and the code should be manually checked to ensure the data obtained is correctly validated and/or sanitised. Additionally, careful checks/sanitisation should be applied in any situation where the user ma
---	----------	---------------------	--

This is another **false positive** in line (242), as shown below, the code can still work even if the image isn't found which is the main reason we chose to use getClass().getResourceAsStream() instead of FileInputStream(). Way easier to handle.

```
240     ImageView imageView = new ImageView();
241     try {
242         var stream = getClass().getResourceAsStream( name: "/Seats.png");
243
244         if (stream != null) {
245             Image image = new Image(stream);
246             imageView.setImage(image);
247             imageView.setFitHeight(50);
248             imageView.setFitWidth(410);
249         } else {
250             System.out.println("Image not found, continuing without image.");
251         }
252     }
```

3.3 Primitive Data Type Operations (Low)

Warnings related to arithmetic operations on primitive types were detected, another **false positive**, as show below all of these are just regex input validations

```
}else if(!showtimeNumber.matches( regex: "\\d+")) {
```

3.4 Classes Not Declared as Final

Several classes are not declared as final. This is a best practice recommendation, not a security threat.

[illegible]

Security Strengths

- Account lockout mechanism (3 failed attempts).
- Password hashing using BCrypt.
- Role-based access control.
- Action logging (audit trail)
- Use of PreparedStatement in multiple areas.
- Regex input validation to every single input by any user.
- Conditioned add/update/delete methods.

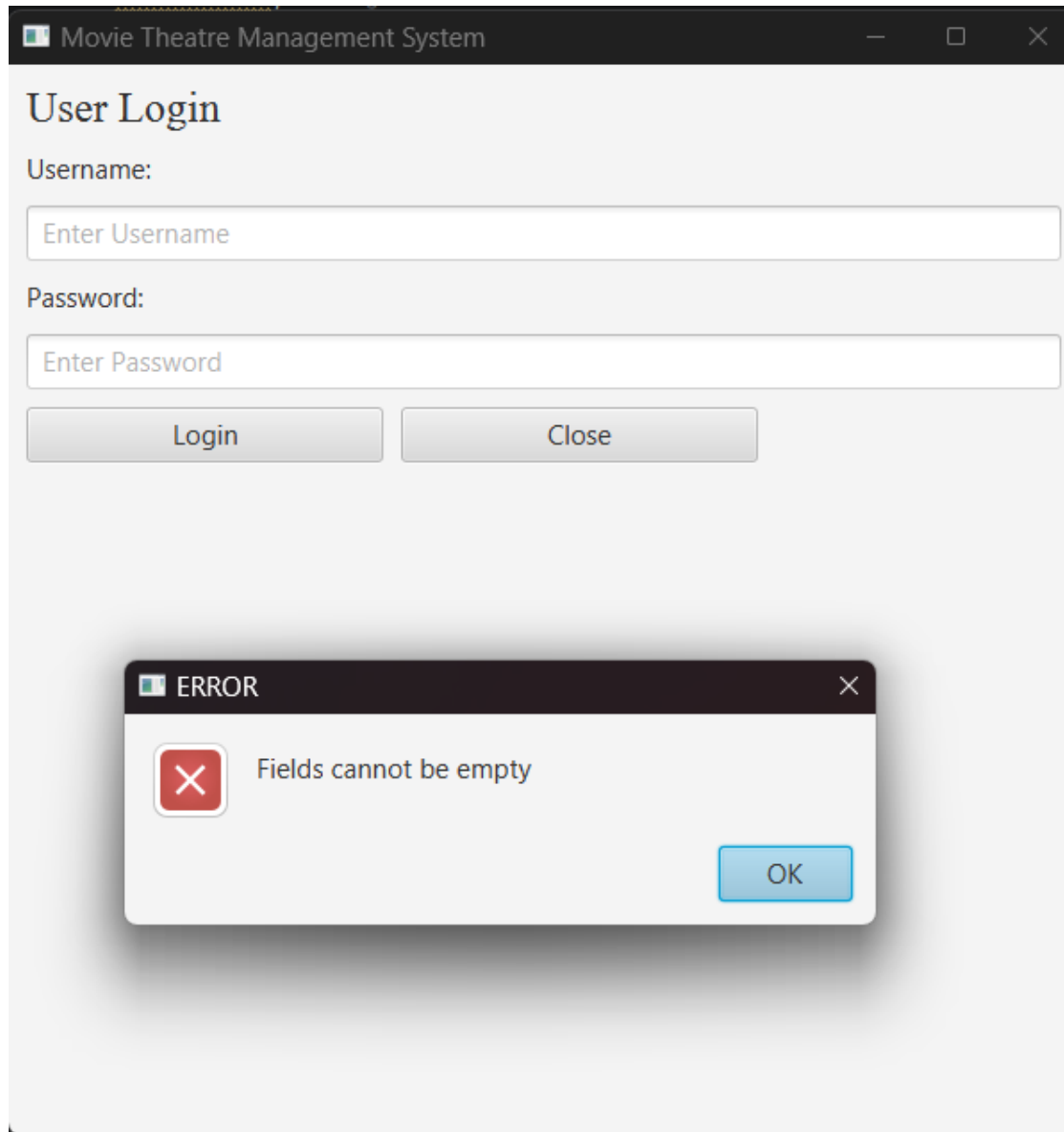
3. DAST

Approach. DAST was done by running the application and trying invalid, unexpected, or misuse-style inputs in important screens. The aim was to see whether the system would block bad input, protect business rules, and stay stable while the app was running.

ID	Feature	Test / Input	Expected Result	Actual Result	Status
DAST-01	Login	Both username and password left empty	Reject the login and show a safe validation message	The app showed “Fields cannot be empty”. It did not crash and nothing unusual appeared in the console.	Pass
DAST-02	Login	Username = ' OR '1'='1, password = 123	Reject the login attempt and prevent bypass	The app showed “Authentication Failed – Invalid username or password”. Only normal connection messages appeared in the console.	Pass
DAST-03	Add Movie	Submitted the Add Movie form with empty fields	Block the insert and show a validation message	The app showed “Movie field is empty”. The insert was blocked and the app stayed normal.	Pass
DAST-04	Add Movie	Tested invalid duration values: abc, 0, and -1	Reject invalid duration values and stop the insert	For all tested values, the app showed “Duration Is Not Valid”. No crash or unusual console output happened.	Pass
DAST-05	Book Ticket	Booked the same seat twice for the same showtime	Allow the first booking and reject the second one	The first booking succeeded. The second attempt showed “Seat Already Taken”.	Pass
DAST-06	Delete Movie	Tried deleting a movie with a showtime, then a movie with no showtime	Block deletion if a showtime exists and allow deletion if no showtime exists	A movie with a showtime was not deleted, while a movie with no showtime was deleted successfully.	Pass
DAST-07	Delete Showtime	Tried deleting a showtime with booked tickets, then one with no tickets	Block deletion if tickets exist and allow deletion if no tickets exist	When tickets existed, the app showed “Cant delete showtime, tickets already exist”. When no tickets existed, it showed “Showtime Deleted successfully”.	Pass
DAST-08	Add Showtime	Entered a non-existing room number	Reject the value and stop the insert	The app showed “Room does not exist” and blocked the showtime.	Pass
DAST-09	Update Movie	Changed duration to an invalid value during update	Reject the invalid update and keep the original data safe	The app showed “Duration Is Not Valid” and blocked the update.	Pass

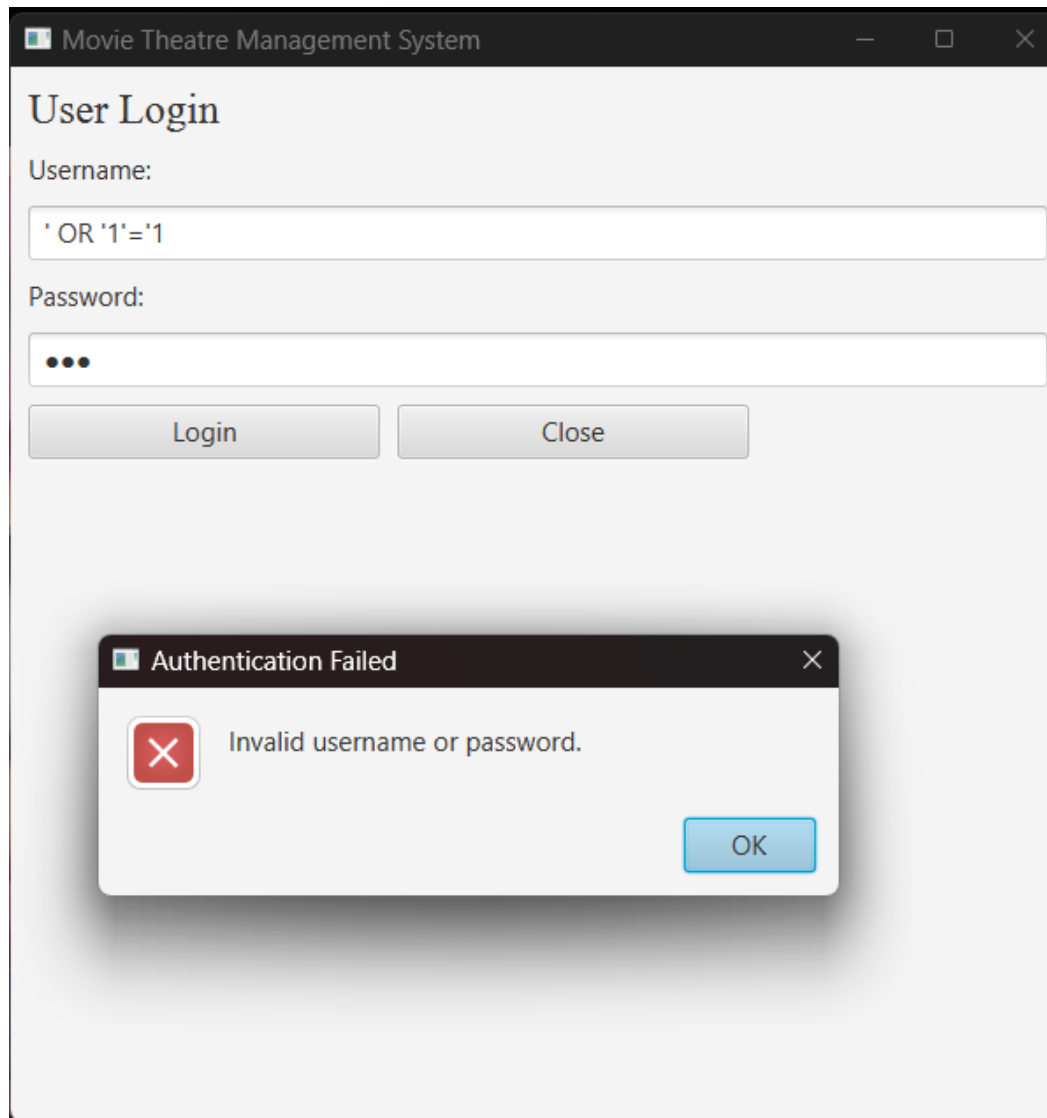
Evidence notes for screenshots

DAST-01.



shows the login screen after pressing Login with both fields empty. The validation alert confirms that empty credentials are blocked safely.

DAST-02.



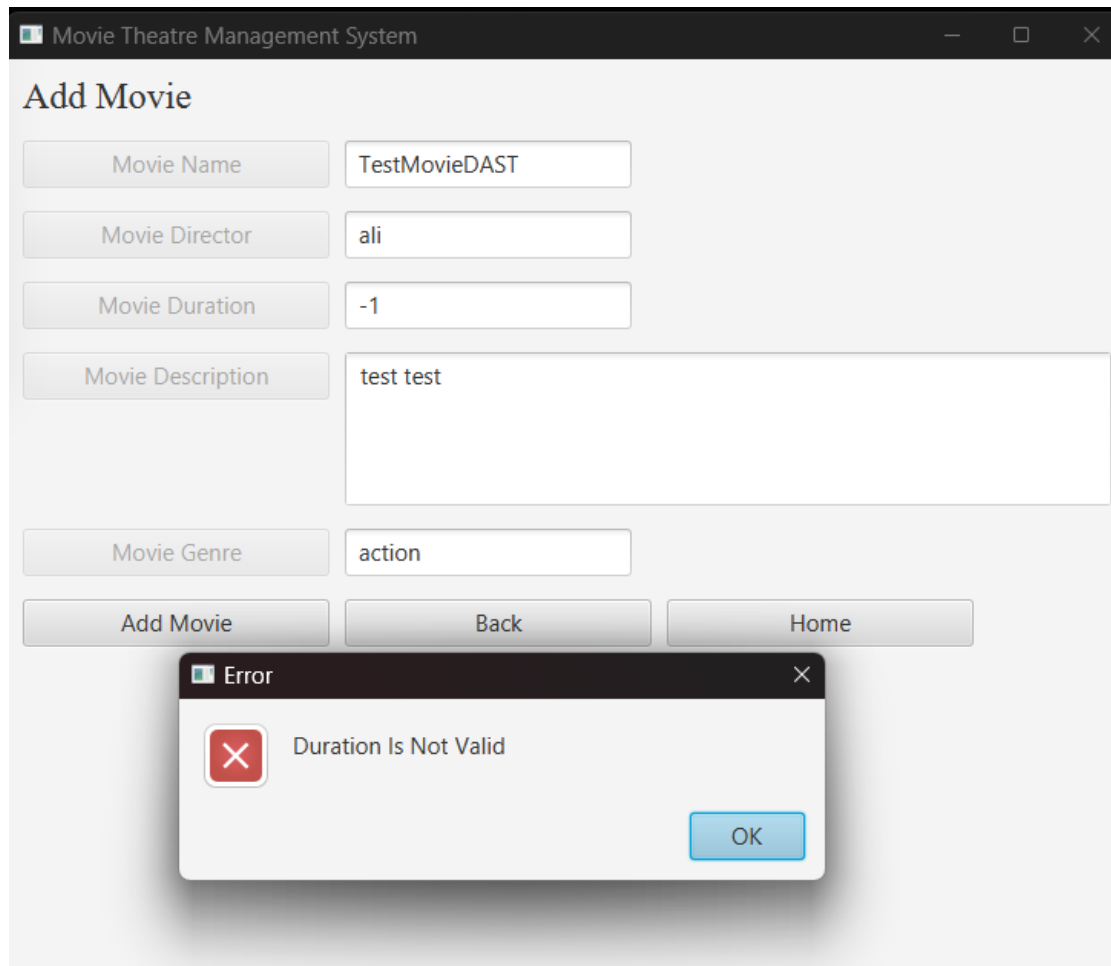
SQL injection-style login attempt being rejected with an authentication failure message

DAST-03.

The screenshot displays a web application window titled "Movie Theatre Management System". Inside, there is a form titled "Add Movie". The form contains several input fields, each with a label button to its left: "Movie Name", "Movie Director", "Movie Duration", "Movie Description" (a larger text area), and "Movie Genre". At the bottom of the form are three buttons: "Add Movie", "Back", and "Home". An error dialog box is overlaid on the bottom half of the screen. The dialog has a title bar that says "Error" and a close button. It features a red square icon with a white "X" and the text "Movie field is empty". An "OK" button is located at the bottom right of the dialog.

shows the Add Movie screen rejecting an empty submission with the message “Movie field is empty”

DAST-04.



The screenshot shows a web application window titled "Movie Theatre Management System". Inside, there is a form titled "Add Movie". The form contains several input fields and buttons. The "Movie Name" field contains "TestMovieDAST", "Movie Director" contains "ali", "Movie Duration" contains "-1", "Movie Description" contains "test test", and "Movie Genre" contains "action". At the bottom of the form are three buttons: "Add Movie", "Back", and "Home". An error dialog box is open in the foreground, titled "Error", with a red "X" icon and the message "Duration Is Not Valid". The dialog has an "OK" button.

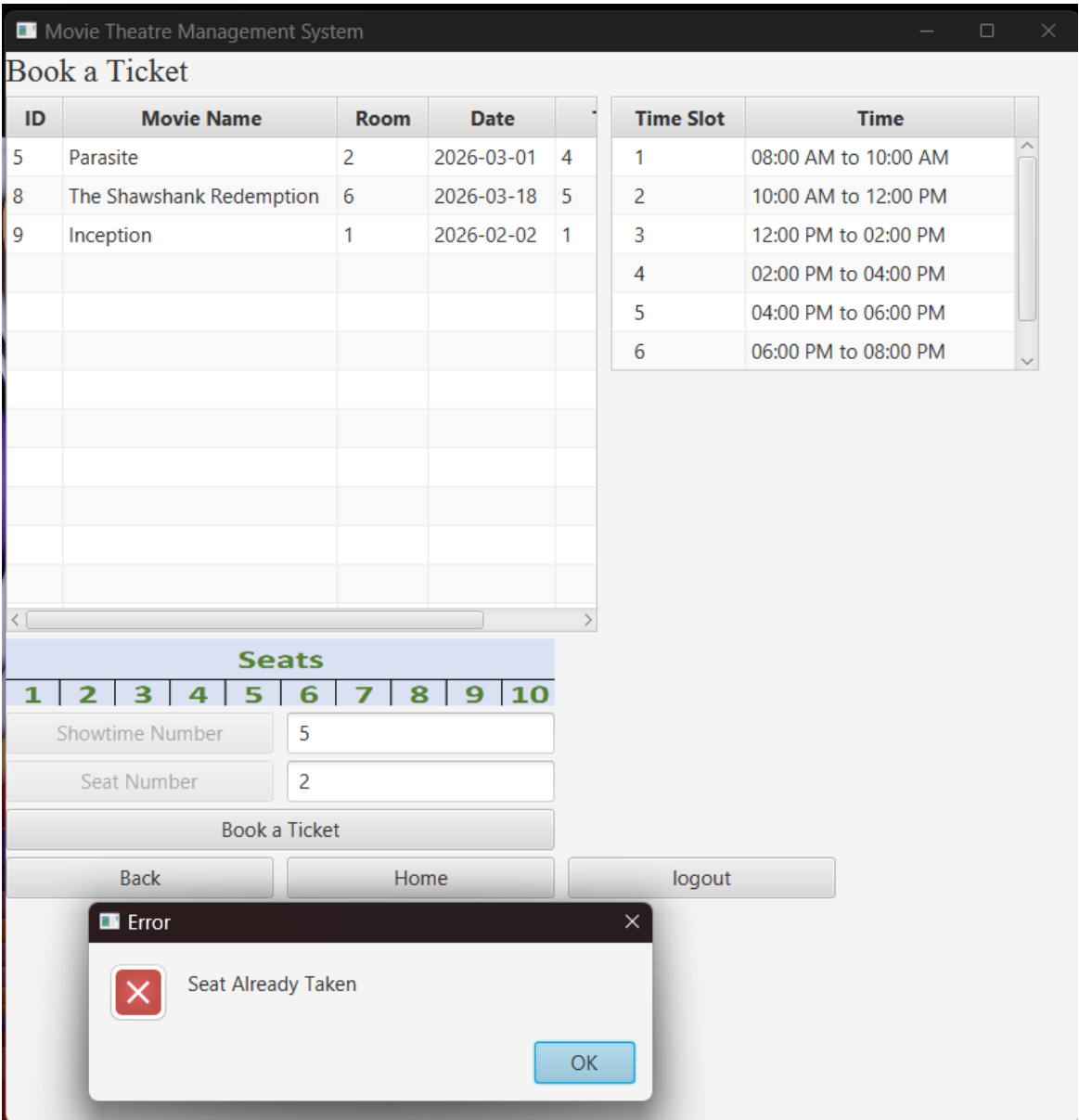
Field Label	Value
Movie Name	TestMovieDAST
Movie Director	ali
Movie Duration	-1
Movie Description	test test
Movie Genre	action

Buttons: Add Movie, Back, Home

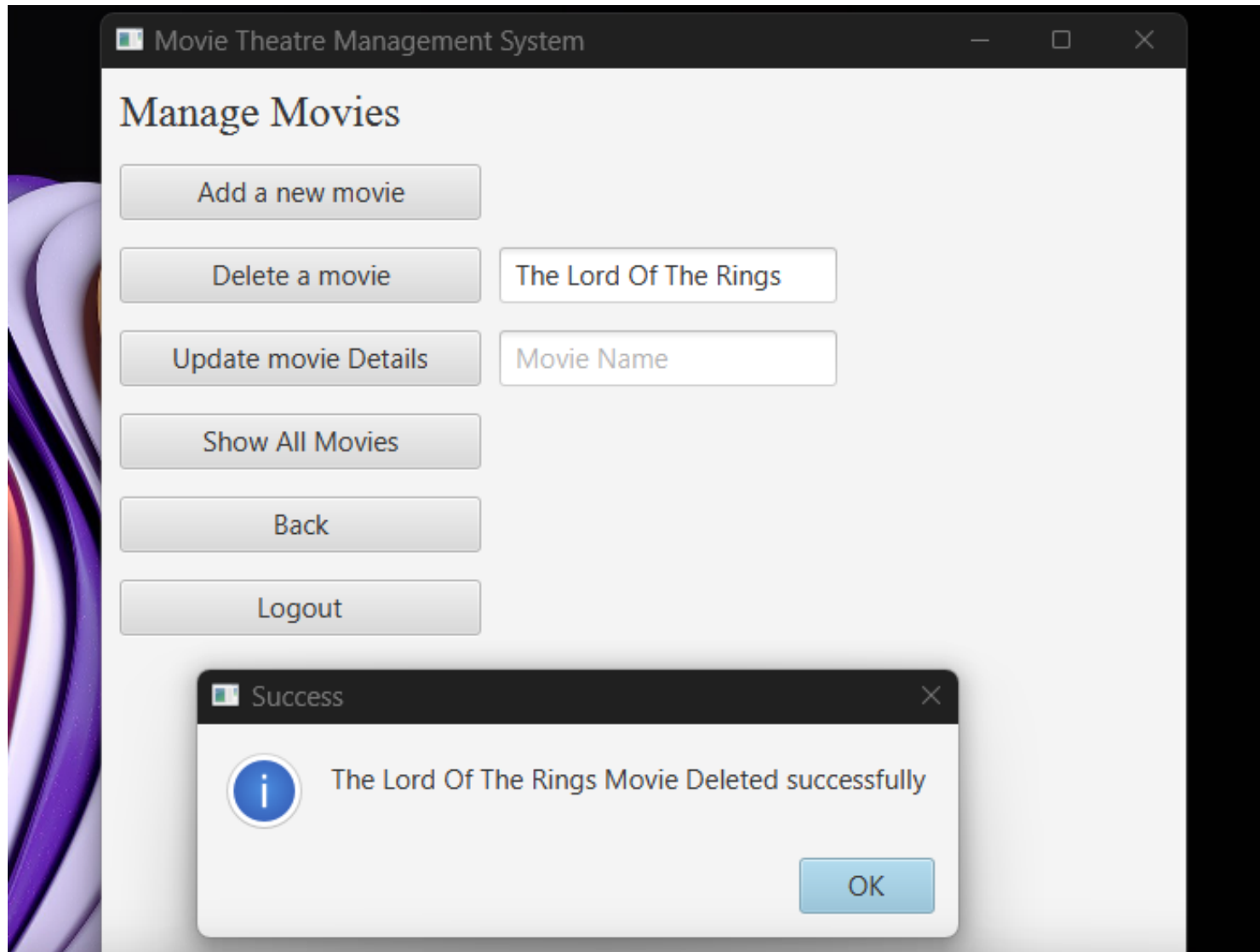
Error Dialog: Duration Is Not Valid (OK)

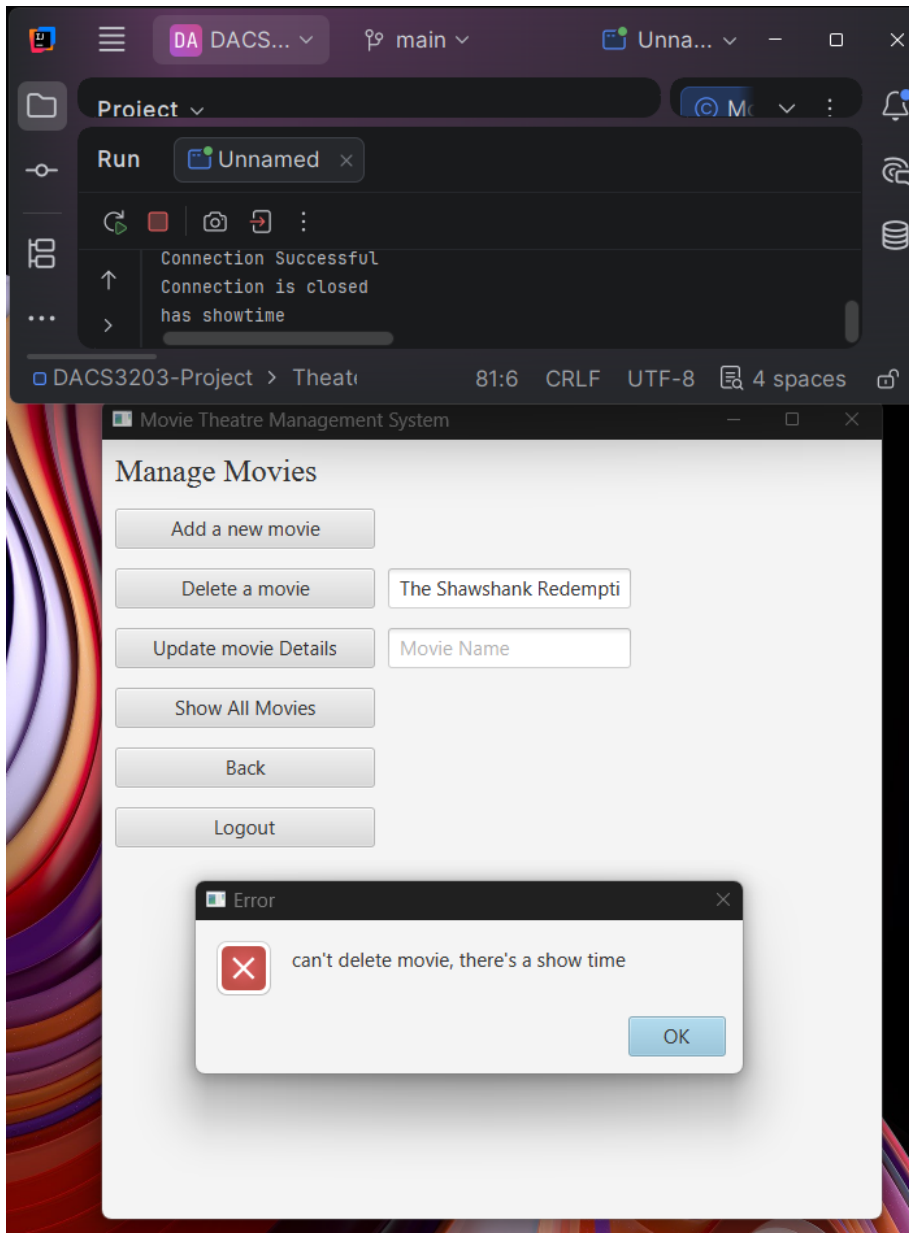
shows Add Movie rejecting an invalid duration value with the message “Duration Is Not Valid”.

DAST-05.



shows the first ticket booking succeeding. Screenshot 2 shows the repeated booking attempt being blocked with the message “Seat Already Taken”

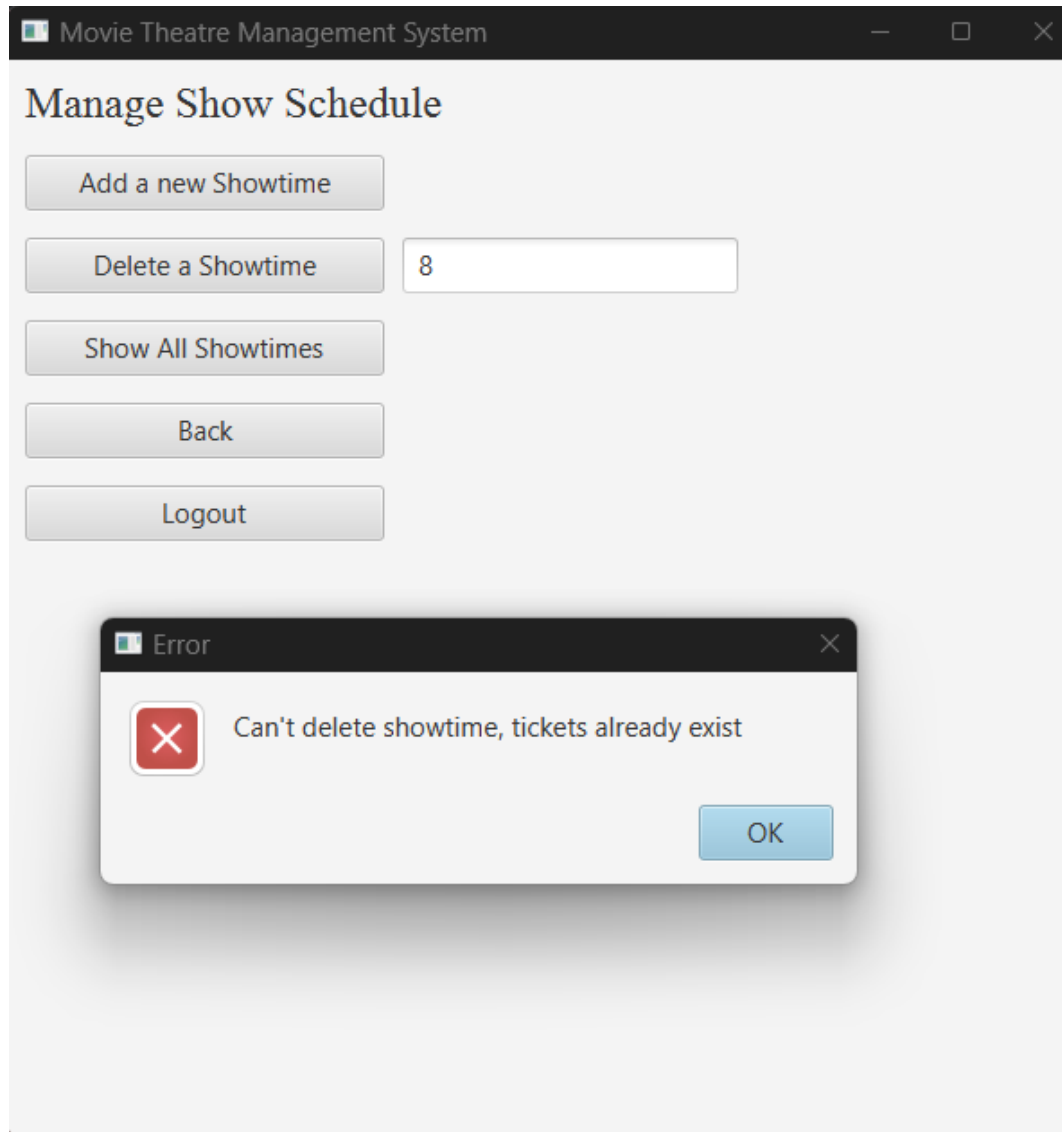


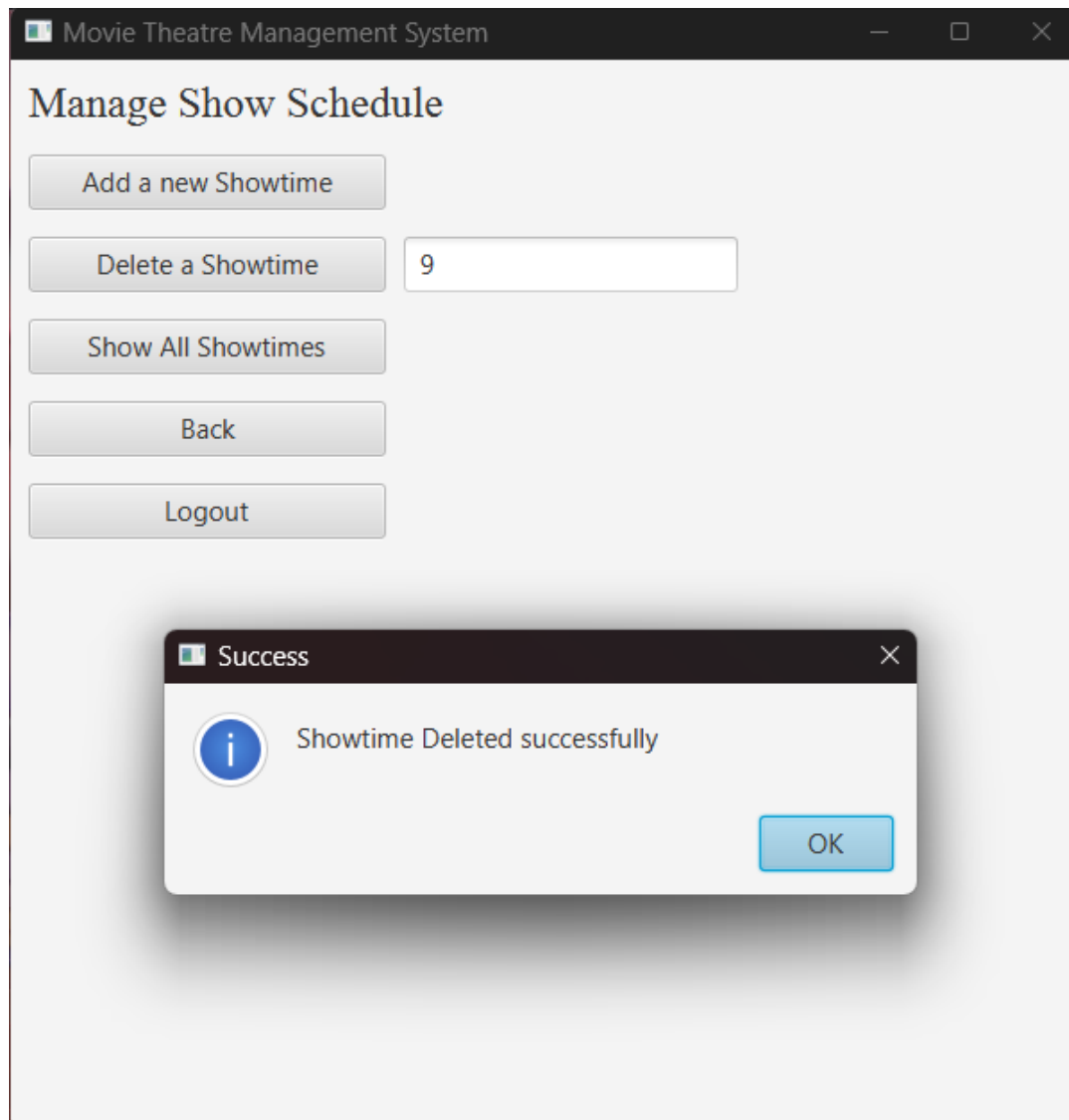


First screenshot shows that a movie without a showtime was deleted successfully.

Second screenshot shows that a movie with a linked showtime could not be deleted

DAST-07.





Screenshot 1 shows showtime deletion being blocked because tickets already exist. Screenshot 2 shows valid showtime deletion succeeding when no tickets are linked

DAST-08.

Movie Theatre Management System

Add Showtime

Movie Name: The Shawshank Redem...

Room Number: 7

Showtime Date: 2026-03-03

Time Slot: 3

Time Slot	Time
1	08:00 AM to 10:00 AM
2	10:00 AM to 12:00 PM
3	12:00 PM to 02:00 PM
4	02:00 PM to 04:00 PM
5	04:00 PM to 06:00 PM
6	06:00 PM to 08:00 PM

Buttons: Add Showtime, Back, Home

Error

Room does not exist

OK

Add Showtime rejecting a room number that does not exist

DAST-9.

The screenshot shows a web application window titled "Movie Theatre Management System". Inside, there is a form titled "Update Movie Details". The form has several input fields and buttons. The "Movie Name" field contains "The Shawshank Redempti". The "Movie Director" field contains "Frank Darabont". The "Movie Duration" field contains "abc". The "Movie Description" field contains "A banker sentenced to life in prison forms a friendship and finds hope while enduring harsh conditions". The "Movie Genre" field contains "Drama". At the bottom of the form are three buttons: "Update Movie", "Back", and "Home". Overlaid on the bottom of the form is an "Error" dialog box. The dialog box has a red "X" icon and the text "Duration Is Not Valid". There is an "OK" button at the bottom right of the dialog box.

Field Label	Value
Movie Name	The Shawshank Redempti
Movie Director	Frank Darabont
Movie Duration	abc
Movie Description	A banker sentenced to life in prison forms a friendship and finds hope while enduring harsh conditions
Movie Genre	Drama

Buttons: Update Movie, Back, Home

Error Dialog: Duration Is Not Valid (OK)

DAST summary. All of the tested validations and business rules worked correctly. Empty fields, invalid values, repeated bookings, and restricted delete actions were all handled properly by the running application.